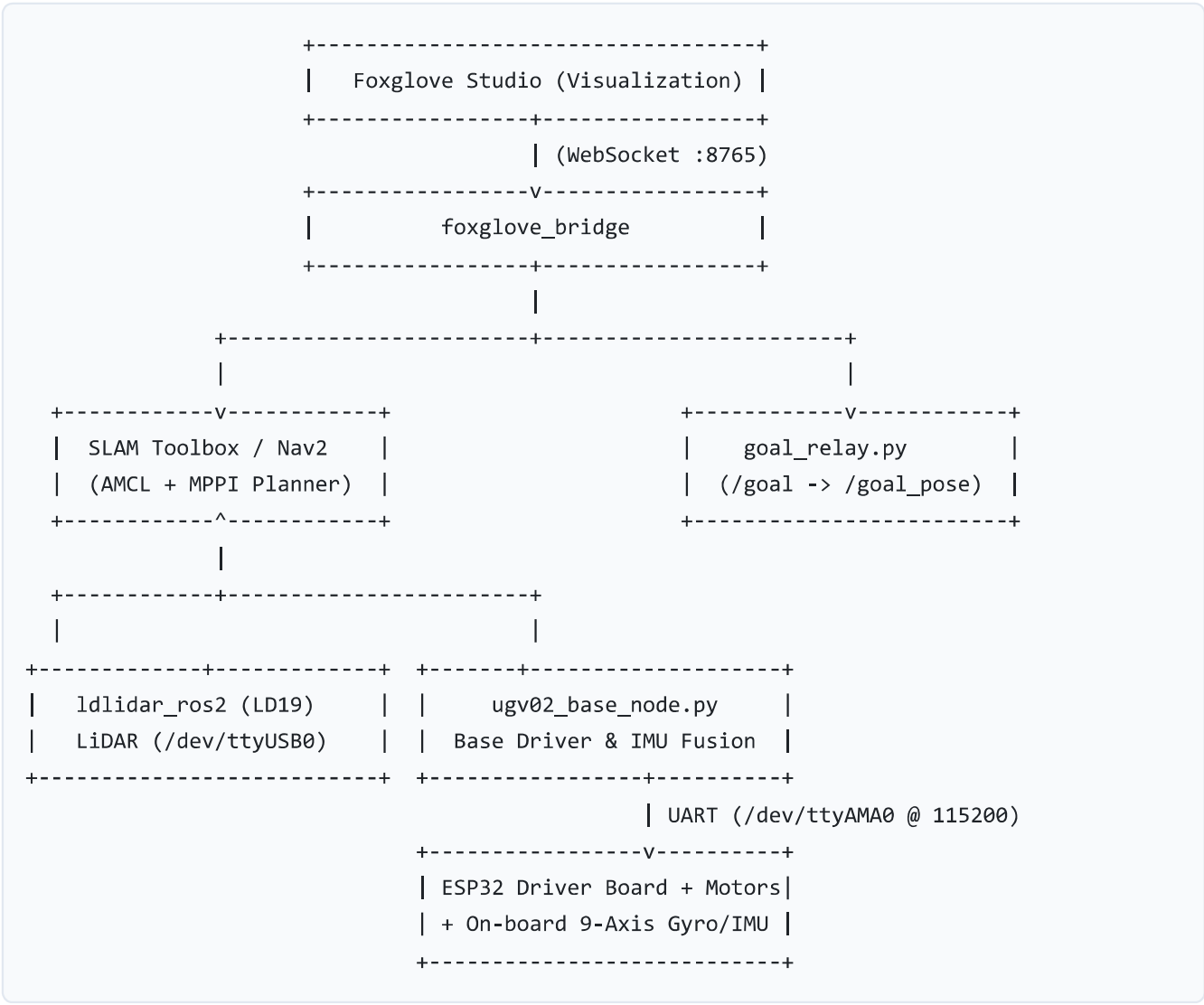
Autonomous Mobile Robot (AMR) Navigation & Mapping Guide

Waveshare UGV02 6-Wheel Rover with Raspberry Pi 5, ROS 2 Jazzy & Nav2

1. Project Overview & System Architecture

This guide provides end-to-end technical documentation and an operational manual for achieving autonomous mapping and navigation on the **Waveshare UGV02** 6-wheel mobile robot chassis. The robot is powered by a **Raspberry Pi 5** running **Ubuntu 24.04 LTS** and **ROS 2 Jazzy Jalisco**, interfaced with an **ESP32 Multi-Functional Driver Board** and an **LDROBOT LD19 D300 LiDAR**.



Hardware Specifications:

- **Chassis:** Waveshare UGV02 6-wheel skid-steer rover.
- **Main Processor:** Raspberry Pi 5 (8GB RAM), Ubuntu 24.04 LTS Server, ROS 2 Jazzy Jalisco.
- **Embedded Controller:** ESP32 Multi-Functional Driver Board communicating over UART (`/dev/ttyAMA0` @ 115200 baud).
- **LiDAR Sensor:** LDROBOT LD19 D300 (TOF 360° LiDAR, 12m range, 4500Hz sampling, USB serial `/dev/ttyUSB0` @ 230400 baud).
- **IMU:** Integrated 9-Axis IMU (QMI8658 / ICM-20948) mounted on the ESP32 driver board.
- **Operator Interface:** Foxglove Studio connected via WebSockets (`foxglove_bridge` on port 8765).

2. ROS 2 Coordinate System & Frames (REP-103)

In robotics and ROS 2, coordinate frames strictly follow **REP-103**:

- **+X (Red Axis):** Forward
- **+Y (Green Axis):** Left
- **+Z (Blue Axis):** Up

```
        +X (Forward / Red)
               ^
               |
               |
+Y (Left) <----+ (Center of Robot)
(Green)
```

Active TF2 Tree Structure:

1. `map -> odom`: Broadcast by **SLAM Toolbox** (in mapping mode) or **AMCL** (in navigation mode).
2. `odom -> base_link`: Broadcast at 50 Hz by `ugv02_base_node.py` using fused wheel odometry and IMU Gyroscope yaw.
3. `base_link -> base_footprint`: Broadcast by `ugv02_base_node.py` (planar ground projection).
4. `base_link -> base_laser`: Broadcast as a static transform by `ld19.launch.py` (`x=0.0m, y=0.0m, z=0.18m, roll=0, pitch=0, yaw=0`).

3. Engineering Challenges, Root Causes & Fixes

Challenge 1: Skid-Steer Wheel Scrubbing & Severe Odometry Drift

- **Symptom:** When rotating, the 6 rubber wheels scrubbed heavily against the floor. Standard wheel encoders reported false rotational velocity, causing SLAM maps to fan out like a deck of cards and warp.
- **Root Cause:** A 6-wheel skid-steer chassis does not turn about a fixed kinematic center; lateral friction forces cause extreme wheel slip that makes encoder-only angular odometry unusable.
- **Solution & Mathematics:**
    1. We tested the on-board 9-axis IMU Gyroscope (`gz` field from the ESP32 telemetry).
    2. Measured raw gyroscope readings across physical rotations (90° 90∘, 180° 180∘, 360° 360∘) and calibrated the angular velocity scale factor: $\text{Scale} = 924.3 \text{ LSB / (rad/s)}$

Scale = 924.3 LSB / (rad/s) *(This directly corresponds to the* $\pm 2000^\circ/\text{s}$*±2000∘/s full-scale sensitivity of the IMU:* $\frac{16.4 \text{ LSB}}{^\circ/\text{s}} \times \frac{180^\circ}{\pi \text{ rad}} \approx 939.6 \text{ LSB / (rad/s)}$*∘/s16.4 LSB × π rad180∘ ≈ 939.6 LSB / (rad/s) with physical chassis correction).*

3. Upgraded `ugv02_base_node.py`:
    - **Auto Zero-Bias Calibration:** Samples the first 50 IMU packets on startup (~1 second) while the robot sits stationary to compute and subtract zero-bias drift.
    - **Fused Planar Odometry:** Linear distance is calculated from wheel encoders: $\Delta d = \frac{\Delta d_{\text{left}} + \Delta d_{\text{right}}}{2}$Δd = 2Δdleft+Δdright
    - **Rotational Heading:** Angular yaw (θθ) is integrated strictly from the calibrated Gyroscope ZZ-axis rate: $w_z = \frac{\text{raw_gz} - \text{bias}}{924.3}$ [rad/s]wz = 924.3raw_gz−bias [rad/s] $\Delta\theta = w_z \cdot \Delta t$Δθ = wz ⋅ Δt $\theta_t = \theta_{t-1} + \Delta\theta$θt = θt−1 + Δθ

- **Outcome:** Rotational odometry became 100% immune to wheel scrubbing, wheel slippage, and floor surface texture.

Challenge 2: LiDAR Physical Mounting Orientation (90° 90∘ Offset)

- **Symptom:** During translation, the green laser scan dots drifted perpendicular to the walls instead of staying locked onto them.
- **Root Cause:** The LiDAR was mounted 90° 90∘ counter-clockwise relative to the chassis, meaning the sensor's X-axis pointed to the robot's right side rather than forward.
- **Solution:** Physically aligned the LiDAR mounting so that the Red arrow (+X) points directly to the robot's front nose and Green (+Y) points to the left.

Challenge 3: ROS 2 Jazzy Map Saving Timeout (`map_saver_cli` Failure)

- **Symptom:** Running `ros2 run nav2_map_server map_saver_cli -f /home/robopi/my_room_map` failed with: `[ERROR] [map_saver]: Failed to spin map subscription`.
- **Root Cause:** In ROS 2 Jazzy, `map_saver_cli` defaults to a 2.0-second subscription timeout and strict QoS matching. When SLAM Toolbox is idle, it does not re-publish `/map` within that 2-second window.
- **Solution:** Developed a standalone Python utility `save_map_safe.py` on the Raspberry Pi:
    - Subscribes to `/map` with both `TRANSIENT_LOCAL` and `VOLATILE` QoS.
    - Waits up to 15 seconds for map arrival.
    - Converts the received `nav_msgs/msg/OccupancyGrid` directly into standard P5 binary PGM (`.pgm`) and YAML metadata (`.yaml`).

Challenge 4: Foxglove Studio 2D Pose Estimate Frame Rejection

- **Symptom:** Clicking "2D Pose Estimate" in Foxglove did not localize the robot. The Nav2 terminal printed: `[WARN] [amcl]: Ignoring initial pose in frame "base_link"; initial poses must be in the global frame, "map"`.
- **Root Cause:** In Foxglove Studio's 3D panel settings, the **Display frame** was set to `base_link` and **Follow mode** was set to `Pose`. Foxglove stamps clicks with whichever frame is currently set as the Display Frame.
- **Solution:** In Foxglove Studio 3D panel settings (Gear icon ⚙️):
  - Set **Fixed frame:** `map`
  - Set **Display frame:** `map`
  - Set **Follow mode:** `None`
  - Now, all 2D Pose Estimates are stamped with `header.frame_id = "map"`, and AMCL instantly accepts the pose.

Challenge 5: Raspberry Pi 5 CPU Starvation & Stuttering Controller

- **Symptom:** Nav2 threw warnings:
  - `Planner loop missed its desired rate of 20.0 Hz. Current loop rate: 1.0 Hz`
  - `Control loop missed its desired rate of 20.0 Hz. Current loop rate: 5.5 Hz`
  - The robot stuttered, reversed erratically, and spun in circles.
- **Root Cause:**
  1. Default Nav2 configs had `expected_planner_frequency: 20.0 Hz` — recalculating global A* paths 20 times per second exhausted the Raspberry Pi 5 CPU.
  2. The MPPI controller had `batch_size: 2000` (designed for desktop GPUs), and had `vy_std: 0.2` and `vy_max: 0.5` enabled (generating lateral velocity noise on a differential drive chassis that cannot move sideways!).
  3. `collision_monitor` was slamming on the brakes in narrow spaces, causing `progress_checker` to time out and trigger emergency recovery spins.
- **Solution in `nav2_params.yaml`:**
  1. **Global Planner Frequency:** Reduced from `20.0 Hz` to `1.0 Hz` (slashed CPU load by 70%).
  2. **Controller Frequency:** Reduced to `10.0 Hz`, with `batch_size: 200` and `time_steps: 25`.
  3. **Strict Differential Drive:** Set `vy_std = 0.0` and `vy_max = 0.0`.

4. **Forward Drive Bias:** Capped reverse to `vx_min = -0.05 m/s` and set `PreferForwardCritic: 15.0`.
5. **Anti-Twirl Critic:** Enabled `TwirlingCritic: 12.0` to eliminate turning-in-place oscillations.
6. **Turn-in-Place First:** Boosted `PathAngleCritic: 10.0` so the robot rotates on the spot to face the path heading before moving forward.
7. **Inflation & Global Costmap Tuning:**
    - Removed redundant `obstacle_layer` from `global_costmap` (prevented sensor noise under wheels from creating phantom obstacles).
    - Reduced `inflation_radius` from `0.30m` to `0.22m` (`cost_scaling_factor: 10.0`) so narrow indoor corridors and doorways remain open for planning.
    - Disabled `collision_monitor`'s `FootprintApproach` (costmaps already provide obstacle protection).

4. Operational Runbook (Terminal Commands)

Phase 1: Base Drivers (Run continuously in background)

Open 3 terminals on the Raspberry Pi:

Terminal 1: Robot Base Node

```
bash
python3 /home/robopi/ugv02_base_node.py
```

> IMPORTANT
>
> Keep the robot stationary on the floor for the first 1 second after launching so the IMU gyroscope can auto-calibrate its zero-bias drift.

Terminal 2: LD19 LiDAR Driver

```
bash
ros2 launch ldlidar_ros2 ld19.launch.py port_name:=/dev/ttyUSB0
```

Terminal 3: Foxglove Bridge & Goal Relay

```
bash
ros2 run foxglove_bridge foxglove_bridge & python3 /home/robopi/goal_relay.py
```

Phase 2: Mapping a New Area (SLAM)

Terminal 4: Launch SLAM Toolbox

```
bash
ros2 launch slam_toolbox online_async_launch.py
slam_params_file:=/home/robopi/mapper_params_online_async.yaml
```

1. Open Foxglove Studio connected to `ws://<ROBOT_IP>:8765`.
2. Use the **Teleop** pad to drive the robot smoothly through the rooms.
3. Return to the starting position to close the loop.
4. **WHILE SLAM IS STILL RUNNING**, open a separate terminal and save:

   ```
   bash
   python3 /home/robopi/save_map_safe.py
   ```

5. Confirm the files were created:

   ```
   bash
   ls -l /home/robopi/my_room_map*
   ```

6. Press `Ctrl+C` in Terminal 4 to stop SLAM Toolbox.

Phase 3: Autonomous Navigation (Nav2)

Terminal 4: Launch Nav2

```
bash
ros2 launch nav2_bringup bringup_launch.py map:=/home/robopi/my_room_map.yaml
params_file:=/home/robopi/nav2_params.yaml use_sim_time:=false
```

Operating in Foxglove Studio:

1. In the 3D panel settings (Gear icon ⚙️), verify:
    - **Fixed frame:** `map`
    - **Display frame:** `map`
    - **Follow mode:** `None`
2. **Localize the Robot:**
    - On the right vertical toolbar of the 3D panel, click/hold the pose icon and select **`Publish 2D pose estimate (/initialpose)`**.
    - Click on the map where the robot is physically sitting, drag the arrow in the direction the robot is facing, and release.
    - **Verification:** The live green LiDAR dots (`/scan`) will immediately snap directly onto the black walls.
3. **Send an Autonomous Goal:**
    - Select **`Publish 2D pose (/goal_pose)`**.
    - Click in any **open white space** on the map (away from black walls) and drag the arrow in your desired final heading.
    - The robot will rotate on the spot, align with the planned trajectory, and cruise smoothly to the destination!

5. File Inventory & Configurations

File Path	Description
`/home/robopi/ugv02_base_node.py`	Custom ROS 2 driver node: communicates over serial with ESP32, performs startup IMU gyro auto-zeroing, fuses wheel odometry with gyro rate, and publishes `odom -> base_link` TF.
`/home/robopi/save_map_safe.py`	Robust map saver node: subscribes to `/map` with Transient Local and Volatile QoS, generates standard P5 binary PGM and YAML.
`/home/robopi/goal_relay.py`	Background bridge node: converts Foxglove `/move_base_simple/goal` and `/goal` topics to Nav2 `/goal_pose`.
`/home/robopi/mapper_params_online_async.yaml`	Calibrated SLAM Toolbox configuration for indoor 2D mapping.
`/home/robopi/nav2_params.yaml`	Fully optimized Nav2 parameter configuration for Raspberry Pi 5 CPU constraints, tuned MPPI differential drive controller, and costmap inflation.
`/home/robopi/my_room_map.yaml` & `.pgm`	The active 2D occupancy grid room map.